

1.Transfer Learning – Plant Disease Detection

```
import tensorflow as tf

from tensorflow.keras import layers, models

from tensorflow.keras.applications import MobileNetV2

base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)

base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(4, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
```

```

        layers.RandomRotation(0.2),

        layers.RandomZoom(0.2),

        layers.RandomContrast(0.2)

    ])

print("Transfer learning model created successfully.")

print("Pre-trained CNN layers are frozen.")

print("Data augmentation techniques are applied.")

```

2.DenseNet + Pixel-wise Segmentation

```

import tensorflow as tf

from tensorflow.keras import layers, Model

from tensorflow.keras.applications import DenseNet121

inputs = layers.Input(shape=(224, 224, 3))

base_model = DenseNet121(

    weights="imagenet",

    include_top=False,

    input_tensor=inputs

)

base_model.trainable = False

x = base_model.output

x = layers.Conv2D(256, 3, padding="same", activation="relu")(x)

x = layers.UpSampling2D((2, 2))(x)

x = layers.Conv2D(128, 3, padding="same", activation="relu")(x)

x = layers.UpSampling2D((2, 2))(x)

```

```

x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)

x = layers.UpSampling2D((2, 2))(x)

x = layers.UpSampling2D((2, 2))(x)

x = layers.UpSampling2D((2, 2))(x)

outputs = layers.Conv2D(
    10,
    1,
    padding="same",
    activation="softmax"
)(x)

model = Model(inputs, outputs)

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

print("DenseNet + Pixel-wise segmentation model created successfully.")

```

3. Bidirectional RNN Encoder–Decoder

```

import tensorflow as tf

from tensorflow.keras import layers, Model

vocab_size = 10000

embedding_dim = 128

```

```

units = 256

max_length = 20

encoder_inputs = layers.Input(
    shape=(max_length,),
    name="encoder_input"
)

encoder_embedding = layers.Embedding(
    vocab_size,
    embedding_dim
)(encoder_inputs)

encoder_lstm = layers.Bidirectional(
    layers.LSTM(
        units,
        return_sequences=True,
        return_state=True
    )
)

encoder_output, forward_h, forward_c, backward_h, backward_c = \
    encoder_lstm(encoder_embedding)

state_h = layers.Concatenate()([forward_h, backward_h])
state_c = layers.Concatenate()([forward_c, backward_c])

decoder_inputs = layers.Input(
    shape=(max_length,),
    name="decoder_input"
)

```

```

decoder_embedding = layers.Embedding(
    vocab_size,
    embedding_dim
)(decoder_inputs)
decoder_lstm = layers.LSTM(
    units * 2,
    return_sequences=True
)
decoder_output = decoder_lstm(
    decoder_embedding,
    initial_state=[state_h, state_c]
)
outputs = layers.Dense(
    vocab_size,
    activation="softmax"
)(decoder_output)
model = Model(
    [encoder_inputs, decoder_inputs],
    outputs
)
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

```

```
model.summary()

print("Bidirectional RNN Encoder-Decoder created successfully.")
```

4.LSTM + BPTT for Stock Trend Prediction

```
import numpy as np

import tensorflow as tf

from tensorflow.keras import Sequential

from tensorflow.keras.layers import LSTM, Dense, Dropout

np.random.seed(42)

data = np.random.rand(500, 1)

time_steps = 10

X = []

y = []

for i in range(len(data) - time_steps):

    X.append(data[i:i + time_steps])

    y.append(data[i + time_steps])

X = np.array(X)

y = np.array(y)

print("Input shape:", X.shape)

print("Output shape:", y.shape)

model = Sequential([

    LSTM(

        64,

        return_sequences=True,
```

```

        input_shape=(time_steps, 1)

    ),
    Dropout(0.2),
    LSTM(32),
    Dense(16, activation="relu"),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.summary()

model.fit(
    X,
    y,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)

print("LSTM stock trend prediction completed.")

```

5.Transfer Learning + LSTM Video Caption Generation

```

import tensorflow as tf

from tensorflow.keras import layers, Model

```

```
from tensorflow.keras.applications import MobileNetV2
```

```
cnn = MobileNetV2(  
    weights="imagenet",  
    include_top=False,  
    pooling="avg",  
    input_shape=(224, 224, 3)
```

```
)
```

```
cnn.trainable = False
```

```
print("CNN feature extractor created.")
```

```
feature_input = layers.Input(  
    shape=(20, 1280),  
    name="video_features"
```

```
)
```

```
encoder = layers.LSTM(  
    256,  
    return_state=True
```

```
)
```

```
encoder_output, state_h, state_c = encoder(  
    feature_input
```

```
)
```

```
vocab_size = 5000
```

```
embedding_dim = 128
```

```
caption_input = layers.Input(  
    shape=(20,),  
    name="caption_input"
```



```

)
embedding = layers.Embedding(
    vocab_size,
    embedding_dim
)(caption_input)
decoder_lstm = layers.LSTM(
    256,
    return_sequences=True
)
decoder_output = decoder_lstm(
    embedding,
    initial_state=[state_h, state_c]
)
output = layers.Dense(
    vocab_size,
    activation="softmax"
)(decoder_output)
model = Model(
    [feature_input, caption_input],
    output
)
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]

```

```
)  
model.summary()  
print("Video caption generation model created successfully.")
```